CYBERGIS — MID-SEMESTER PRACTICAL
BUILD A MISSING OGC WEB SERVICE OVER BHOONIDHI

Bhoonidhi Spatial Catalogue Analysis

An OGC API — Processes service for spatial analysis
over ISRO's national remote sensing catalogue

Name Sambhavi Singh
Roll number 2024117010
Repository https://github.com/sambhavi4/bhoonidhi-records

Date October 12, 2026

What this report covers

The gap	Bhoonidhi stores every scene's footprint polygon and lets you ask one question of it: does it intersect mine? No analysis, no processing endpoint anywhere in ISRO's public stack.
Why it matters	Measured: **208 scenes returned where 16 cover the same ground — 92 % redundant**, 206 GB ordered where 16 GB was needed. Copernicus publishes 146 server-side processes; Bhoonidhi publishes 0.
What was built	A conformant **OGC API — Processes** service (pygeoapi, shapely): six spatial operations, sync + async execution, a job API, areas in an equal-area projection.
Evidence	**100/100 compliance tests pass.** Every number here is produced by a script in the repository and saved under `docs/evidence/`.

Standard implemented: OGC API — Processes, Part 1: Core (OGC 18-062r2) • Supporting: OGC API — Records (OGC 20-004)
Framework: pygeoapi 0.24.0, shapely 2.1, pyproj 3.8, rasterio • Deployment: `http://localhost:5003`

Contents

1 Introduction **3**
1.1 The portal . . . 3
1.2 The task . . . 3
1.3 What this project identifies, and what it builds . . . 3
1.4 How to read this report . . . 4

2 Methodology: how the gap was established **4**
2.1 Method 1 — specification audit (offline-reproducible) . . . 4
2.2 Method 2 — comparative baseline (live) . . . 5
2.3 Method 3 — measuring the cost (live, against this service) . . . 5
2.4 What would falsify the claim . . . 5

3 Gap analysis **6**
3.1 What is missing — the capability audit . . . 6
3.2 This gap is not about authentication . . . 7
3.3 Why this is a large gap, not a small one . . . 7
3.3.1 Argument 1 — an entire standardised tier is absent . . . 7
3.3.2 Argument 2 — every comparable portal has it . . . 7
3.3.3 Argument 3 — the cost is measurable, and it is 92 % . . . 8
3.3.4 Argument 4 — the workaround is expert-only, so nobody does it . . . 9
3.4 Counter-arguments, answered . . . 9
3.5 Scope of the claim . . . 10

4 Service design **10**
4.1 Architecture . . . 10
4.2 The two layers, deliberately . . . 10
4.3 Why these six operations . . . 11
4.4 Measurement: areas in an equal-area projection . . . 11
4.5 Honesty about provenance, in the response . . . 11
4.6 Implementation choices, and why . . . 12

5 Implementation **12**
5.1 Stack and layout . . . 12
5.2 Process structure . . . 13
5.3 Job management and asynchronous execution . . . 13
5.4 Defects found during development, and fixed . . . 14
5.5 Deployment . . . 14

6 OGC compliance evidence **15**
6.1 Conformance classes declared . . . 15
6.2 The mandatory operations . . . 15
6.3 Test results . . . 15
6.4 The suite tests the mathematics, not only the protocol . . . 15
6.5 What this evidence is not . . . 17

7 Live demonstration **17**
7.1 The gap, closed: which scenes do I actually need? . . . 17
7.2 `aoi-coverage` — coverage and explicit gap polygons . . . 18
7.3 `multi-sensor-overlap` — co-coverage for fusion . . . 18
7.4 `revisit-density` — acquisition density grid . . . 18

7.5 The raster tier . . . 18
7.6 Asynchronous execution and the job API . . . 19
7.7 The supporting Records tier . . . 19
7.8 Errors are proper errors . . . 19
7.9 Performance . . . 19

8 Known limitations **20**

9 Conclusion **21**

A Request and response transcripts **23**
A.1 Process list — `GET /processes` . . . 23
A.2 `minimal-scene-set` — full response . . . 23
A.3 `revisit-density` — summary block and one cell . . . 23
A.4 Asynchronous execution — full exchange . . . 24
A.5 The provenance block, present on every response . . . 24
A.6 Gap evidence — `probe_baseline.sh` output . . . 25
A.7 Cost evidence — `measure_redundancy.py` output . . . 25

B Reproducing everything in this report **25**
B.1 Bringing the service up . . . 25
B.2 Reproducing the gap evidence . . . 26
B.3 Building this report . . . 26
B.4 Exposing a public URL . . . 26
B.5 Serving real Bhoonidhi footprints . . . 26
B.6 File map for a reviewer in a hurry . . . 27

1 Introduction

1.1 The portal

Bhoonidhi (`https://bhoonidhi.nrsc.gov.in/`) is the National Remote Sensing Centre's public repository for Indian Earth observation data. It holds satellite imagery, thematic maps and derived products across approximately 59 published collections spanning the ResourceSat, Cartosat, Oceansat, EOS and RISAT missions, together with mirrored international holdings such as Sentinel-1 and Sentinel-2.

Alongside the browser portal, NRSC publishes an *Open Data Access API* [1]. It is worth stating plainly at the outset what that API does well, because an argument that misrepresents its target is worthless:

- it is built on **STAC 1.0.0** [2], a current community standard for Earth observation metadata;
- it has proper token authentication (`/auth/token`, `/auth/logout`);
- `/data/collections` enumerates the published collections;
- `/data/search` accepts a GeoJSON `intersects` geometry, a `bbox`, RFC 3339 `datetime` ranges and CQL2-JSON filters;
- `/download` delivers a product by item identifier.

That is a competent discovery API. A developer can authenticate, search by space and time, and fetch. **Discovery is solved, and this project claims no credit for noticing otherwise.**

1.2 The task

The practical brief asks for one thing: identify a dataset or data product on Bhoonidhi that would genuinely benefit from an OGC service interface which does not currently exist, verify that the gap is real, then build and deploy a conformant service that fills it. Gap identification carries 5 of the 20 marks and is assessed on whether the justification is *convincing*; OGC compliance carries 10; a working deployment carries 5.

1.3 What this project identifies, and what it builds

The gap identified here is not in discovery and not in access control. It is in **computation**.

Bhoonidhi stores the footprint polygon of every scene it holds. Its whole spatial surface, audited against its own published specification, is `bbox` and `intersects` — which are one predicate expressed two ways: *does this item intersect my area of interest?* There is no union, no difference, no area, no aggregation, no statistics, and no processing endpoint anywhere in ISRO's public stack.

The practical consequence is that a user cannot learn *which* of the scenes a search returns they actually need. Lacking that, the rational behaviour — and the observed behaviour — is to order everything the search returned. Section 3.3.3 measures what that costs: across eight queries the catalogue returns 208 scenes where 16 cover the same ground to the same percentage.

This report documents a conformant **OGC API — Processes** service [3] that closes the gap, exposing six operations:

Process	The question it answers
`aoi-coverage`	How much of my AOI do these scenes cover, and where are the holes?
`minimal-scene-set`	Which few of the scenes returned do I actually need?
`multi-sensor-overlap`	Where do optical and radar co-cover, for fusion?
`revisit-density`	How uneven is acquisition density across my AOI?
`zonal-statistics`	What are the raster values inside this polygon?
`scene-raster-profile`	What terrain is actually underneath each candidate scene?

The first four read only footprint geometry — deciding which scenes are worth fetching. The last two read pixels — establishing what is actually inside them. `scene-raster-profile` joins the two tiers. This is deliberately *not* spectral index computation: there is no band maths and no derived imagery product, because the gap being filled is spatial algebra over the catalogue's own geometry.

A supporting **OGC API — Records** [4] catalogue of Bhoonidhi's collections ships alongside the Processes tier. It is infrastructure rather than the gap being filled: the spatial processes need discoverable collection identifiers to point at. The substance of the submission rests on the Processes tier.

1.4 How to read this report

Section 2 states how the gap was established and what counts as evidence. Section 3 is the gap analysis and is the core of the argument. Sections 4 and 5 cover service design and implementation; Section 6 the OGC compliance evidence; Section 7 the live demonstration; Section 8 the limitations, stated plainly. Every number in this report is produced by a script in the repository and saved under `docs/evidence/`; Appendix B lists the commands.

2 Methodology: how the gap was established

A gap claim is only as good as the evidence behind it, and the easiest way to get this wrong is to assert an absence from a cursory look at a portal. Three independent methods were used, each implemented as a script that saves its raw responses so that a reviewer can re-run it and diff the result.

2.1 Method 1 — specification audit (offline-reproducible)

`scripts/probe_gap.sh` audits Bhoonidhi's *own published API specification* rather than probing the live service. It parses the saved specification document and mechanically enumerates:

- which spatial parameters `/data/search` documents;
- which spatial predicates are *absent* — `crosses`, `touches`, `overlaps`, `disjoint`, `s_within` and `s_contains`;
- which CQL2 operations appear in the documented examples;
- whether any analysis or aggregation vocabulary appears anywhere;
- which endpoints exist, and whether any of them computes.

Auditing the specification rather than the service was a deliberate choice with two benefits. First, it is the fairest possible reading: it tests what NRSC says the API does, not what a particular deployment happened to answer on a particular afternoon. Second, it is **reproducible offline** from `docs/evidence/bhoonidhi-api-spec.html` and does not depend on NRSC being reachable — which mattered in practice, because both NRSC hosts stopped answering from the build host partway through development (DNS resolved, connections timed out, the rest of the internet was fine). The `-refresh` flag re-fetches the specification when the hosts are back and says so plainly when it cannot.

The script also deliberately searches for words that *look* like operators but are not. `contains`, `within` and `area` do appear in the specification — as English prose, never as parameters. Testing for this explicitly turns a potential counter-example into part of the argument instead of a hole in it.

2.2 Method 2 — comparative baseline (live)

An audit alone establishes that the gap exists but not that it is *large*. On its own it invites the reply: *server-side geospatial computation is exotic, nobody deploys it, this is an unreasonable thing to expect of a national portal.*

`scripts/probe_baseline.sh` tests that reply against the field. It probes ISRO's public stack for any processing endpoint, records how long the relevant standards have existed, and then probes comparable Earth observation portals live to see whether they actually expose the capability today. All probes are single, unauthenticated, polite `GET`s against published public endpoints. Portals that require registration are documented but deliberately *not* probed, on the grounds that hammering them from a course project would be impolite rather than informative.

2.3 Method 3 — measuring the cost (live, against this service)

The decisive method, because it converts an architectural complaint into a number.

`scripts/measure_redundancy.py` asks this project's own `minimal-scene-set` process the question the catalogue cannot answer, over an area-of-interest ladder from a taluk-sized window to a regional transect, for both an optical and a radar collection. For each it records how many scenes a search would return against how many are genuinely required to cover the same ground to the same percentage.

What is measured and what is modelled. This distinction is kept explicit throughout, including in the script's own console output, because conflating the two would be the easiest way to inflate the result:

- the **scene counts are measured** live against the running service over real STAC footprint geometry. This is the part that carries the argument and it is not modelled;
- the **byte figures are modelled**: published nominal per-scene product sizes multiplied by those counts. The script never downloads imagery. A real order varies with processing level, tiling and compression;
- the ResourceSat figures are a **projection** onto Bhoonidhi's own holdings — the measured footprint geometry is Sentinel-2 and Sentinel-1, so the per-scene volume is swapped for the NRSC product's while the geometry stays what was actually measured. Output labels it as such.

The measurement is a lower bound. The footprint store holds 68 scenes over a three-month window, because that is what was harvested. Several rows in the result are therefore capped by the store rather than by the area of interest. A real Bhoonidhi search spans a multi-decade archive with many revisits per year: it returns far more intersecting scenes for the same AOI while the number genuinely needed to cover it stays roughly constant. Redundancy therefore *rises* with archive depth, and the figure reported in Section 3.3.3 is the conservative end.

2.4 What would falsify the claim

Stated in advance, because a claim that cannot be falsified is not a finding:

- if `/data/search` documented any spatial operator beyond intersection, Method 1 reports `NOT MATCHED`;
- if any ISRO endpoint answered a processing request, Method 2 reports `NOT MATCHED`;
- if the minimal covering set were close in size to the candidate set, Method 3 would return a redundancy near zero and the cost argument would collapse.

Each script prints its own failure loudly rather than quietly passing. If NRSC enables a processing endpoint, the honest response is to say the gap has closed, and the scripts are written so that this is what happens.

3 Gap analysis

The gap, in one sentence

Bhoonidhi stores the footprint polygon of every scene it holds, and the only question you are permitted to ask of that geometry is *"does it touch mine?"*

Everything in this section is either evidence for that sentence or an argument about why it matters more than it sounds like it does.

3.1 What is missing — the capability audit

Method 1 (Section 2) produces Table 1, written to `docs/evidence/capability-audit.json`.

Table 1: Bhoonidhi's spatial and analytical surface, audited against its own published specification.

Question	Answer
Spatial parameters on `/data/search`	`bbox` and `intersects` — and nothing else
What they compute	the same single predicate: *does this item intersect my AOI?*
Other spatial operators documented	absent — no `crosses`, `touches`, `overlaps`, `disjoint`, `s_within`, `s_contains`
CQL2 operations in the documented examples	`eq`, `and` — attribute equality only
Analysis vocabulary anywhere in the API	absent — no union, aggregation, statistics, coverage, density
Endpoints documented	5: authenticate, logout, list, search, download
Endpoints that compute anything	**0**

So the catalogue holds the geometry and will not compute with it. The questions that actually decide an acquisition order are all exactly one step beyond what it offers (Table 2).

Table 2: Ordinary spatial questions, the operations they require, and their availability on Bhoonidhi today. Every input these need is geometry the catalogue already stores.

Question an analyst asks	Operations required	Today
How much of my AOI do these scenes really cover, and where are the holes?	union, difference, area	**no**
Which few of the 30 scenes returned do I actually need?	set cover over geometry	**no**
Where do optical and radar co-cover, for fusion?	intersection of two unions	**no**
How uneven is revisit across my AOI?	gridding, per-cell counts	**no**
What terrain is under each candidate scene?	raster–vector overlay	**no**

3.2 This gap is not about authentication

Stated explicitly and early, because it is the most tempting *wrong* version of this argument.

Requiring a token is normal, sensible practice. Copernicus, USGS EarthExplorer and NASA Earthdata all gate downloads. This project makes no complaint about it whatsoever.

> **The gap stands for a fully authenticated user**
>
> Hand someone a valid JWT and the complete API documentation, and every limitation in Table 1 is still in force. They can learn *which* scenes touch their area, and nothing further. This is a **capability** gap, not an **access** gap.

3.3 Why this is a large gap, not a small one

Four independent arguments. Each is measured rather than asserted.

3.3.1 Argument 1 — an entire standardised tier is absent

Server-side geospatial computation is neither exotic nor speculative. It has been an OGC standard since 2007 (Table 3).

Table 3: Standards for server-side geospatial processing, and Bhoonidhi's implementation status.

Standard	Document	Year	Bhoonidhi
OGC Web Processing Service 1.0.0	05-007r7	2007	no
OGC WPS 2.0.0	14-065	2015	no
OGC API — Processes, Part 1: Core	**18-062r2**	**2021**	no
openEO API (EO process graphs)	community	2020	no

Method 2 probes ISRO's public stack for any of it:

```
Bhuvan WPS 1.0.0 GetCapabilities       HTTP 404   absent
Bhuvan OGC API - Processes landing     HTTP 404   absent
Bhoonidhi /processes                   HTTP 404   absent
  MATCHED  no server-side computation anywhere in the public stack
```

The missing thing is therefore not a query parameter someone forgot. It is the whole processing tier of the OGC stack — the tier that turns a catalogue into something you can compute with — absent from India's national remote sensing portal nearly two decades after it was standardised.

3.3.2 Argument 2 — every comparable portal has it

The strongest objection to Argument 1 is that the standard is a paper standard nobody deploys. Method 2 tests this against the field, live (Table 4).

Table 4: Processing capability at comparable Earth observation portals, probed live and unauthenticated. The lower two rows are ISRO's.

Portal	Endpoint probed	Result
Copernicus Data Space (openEO)	`/openeo/1.2/processes`	**200 — 146 processes**
MS Planetary Computer	STAC `/api/stac/v1/`	200 — 15 classes
Bhoonidhi	`/processes`	**404**
Bhuvan	`/bhuvan/wps`	**404**

Documented but deliberately not probed, each requiring registration: **NASA Earthdata Harmony** [5] (server-side subsetting, reprojection, regridding and aggregation as a service), **USGS ESPA / M2M** (on-demand higher-level Landsat products), **Google Earth Engine** [6] (computation *is* the access model), and **Digital Earth Africa** (Open Data Cube with OGC API — Processes).

> **The contrast**
>
> The Copernicus Data Space publishes **146** server-side processes at a public, unauthenticated endpoint that anyone can enumerate right now. Bhoonidhi publishes **0**. Bhuvan's WPS endpoint returns **404**.
>
> That is not a scoping decision that everyone else also made. It is the single capability that separates Bhoonidhi from every portal it would be compared against.

3.3.3 Argument 3 — the cost is measurable, and it is 92 %

This is the decisive argument.

Because the catalogue will not compute with footprint geometry, a user has no way to learn which of the returned scenes they actually need. So the rational move — and the observed one — is to order everything the search returned. Method 3 measures how much of that is waste (Table 5).

Table 5: Measured redundancy. *found* is what a search hands back; *needed* is what covers the same ground to the same percentage, computed by this service's `minimal-scene-set`. Scene counts are measured; GB figures are published nominal product sizes multiplied by those counts.

AOI	Collection	km²	found	needed	redundant	GB today	GB needed
taluk	sentinel-2-l2a	479	30	1	97 %	30.0	1.0
taluk	sentinel-1-grd	479	4	1	75 %	3.7	0.9
district	sentinel-2-l2a	4 191	30	2	93 %	30.0	2.0
district	sentinel-1-grd	4 191	8	2	75 %	7.4	1.9
two districts	sentinel-2-l2a	11 855	60	2	97 %	60.0	2.0
two districts	sentinel-1-grd	11 855	8	2	75 %	7.4	1.9
regional transect	sentinel-2-l2a	59 816	60	4	93 %	60.0	4.0
regional transect	sentinel-1-grd	59 816	8	2	75 %	7.4	1.9
Total			**208**	**16**	**92 %**	**206.0**	**15.5**

> **The measured cost of the missing tier**
>
> Across 8 queries over 4 areas of interest the catalogue hands back **208 scenes**. **16** of them cover the same ground to the same percentage. **192 are redundant — 92 % of everything returned.**
>
> At published product sizes that is **206 GB ordered where 16 GB was required**: 190 GB of avoidable transfer, for four areas of interest. Projected onto Bhoonidhi's own ResourceSat-2 LISS-III volumes (250 MB per scene), 47 GB.
>
> **And nothing in Bhoonidhi's API lets a user discover the 92 % figure.**

As set out in Section 2, this is a **floor**: several rows are capped by the depth of the footprint store rather than by the area of interest, and redundancy rises with archive depth. A multi-decade archive returns far more intersecting scenes for the same AOI while the number needed to cover it stays roughly constant.

3.3.4 Argument 4 — the workaround is expert-only, so nobody does it

The honest counter-argument is *"you can just do this yourself."* True — and that is precisely the gap. The complete procedure a user must follow today to learn the 92 % figure for their own AOI:

1. authenticate, `POST /data/search`, page through every result;
2. extract each item's footprint geometry from the STAC response;
3. load 30+ polygons into a desktop GIS or a `shapely` script;
4. union them, then difference the union against the AOI to find holes;
5. reproject to an equal-area CRS — or every area number is wrong;
6. implement a **set-cover** optimisation over the footprints;
7. map the chosen geometries back to orderable item identifiers.

Step 6 is an NP-hard combinatorial problem [7]. Steps 4–5 are where most people silently get it wrong, because areas computed in degrees are meaningless. This is a GIS-literate programmer's afternoon, repeated for every area of interest, by every user, indefinitely — and it is the *same seven steps every time*, over geometry the server already holds in memory at the moment it answers the search.

So in practice nobody does it. They download all 30 scenes. This service performs all seven steps in one HTTP request (Section 7).

3.4 Counter-arguments, answered

Taken seriously rather than ignored, because a gap claim that cannot survive its own objections is not convincing.

"Requiring a token is the real barrier." No — Section 3.2. Every limitation here applies to a fully authenticated user, and gating downloads is normal practice.

"The data isn't secret, so there's no gap." Agreed, and the gap is not about secrecy. It is that there is no way to *compute* with the geometry. Section 3.3.3 measures what that costs.

"Server-side processing is a paper standard." Measured against the field in Table 4: Copernicus publishes 146 processes publicly, today, at an endpoint this project enumerates live.

"Users can script it themselves." They can, in seven steps including an NP-hard optimisation and a reprojection that is easy to get silently wrong. The purpose of a service interface is that the server does once, correctly, what every client would otherwise do repeatedly and inconsistently.

"STAC search *is* the spatial API." STAC search is a *filter*: it selects items. It does not perform geometry operations and the specification does not claim to. That is exactly the division of labour OGC API — Processes exists to fill.

"This is a niche optimisation." 92 % of returned scenes are redundant, and the figure rises with archive depth. Order-of-magnitude reductions in transfer volume are not niche, particularly for users on Indian academic and state-government bandwidth, who are the portal's primary audience.

3.5 Scope of the claim

Claimed: Bhoonidhi has no server-side spatial computation; that capability is standardised, deployed by peers, and its absence has a measurable cost; and a conformant OGC API — Processes service over footprint geometry closes it.

Not claimed: that Bhoonidhi is badly built (Section 3 is explicit that discovery is solved), that authentication is wrong, that this project's footprints *are* Bhoonidhi's (they are substituted and flagged — Section 8), or that this is an official NRSC product.

4 Service design

4.1 Architecture

The service is a single pygeoapi instance exposing two OGC API families over two data stores. Figure 1 shows the whole system, including where the data comes from and which part of it stands in for Bhoonidhi.

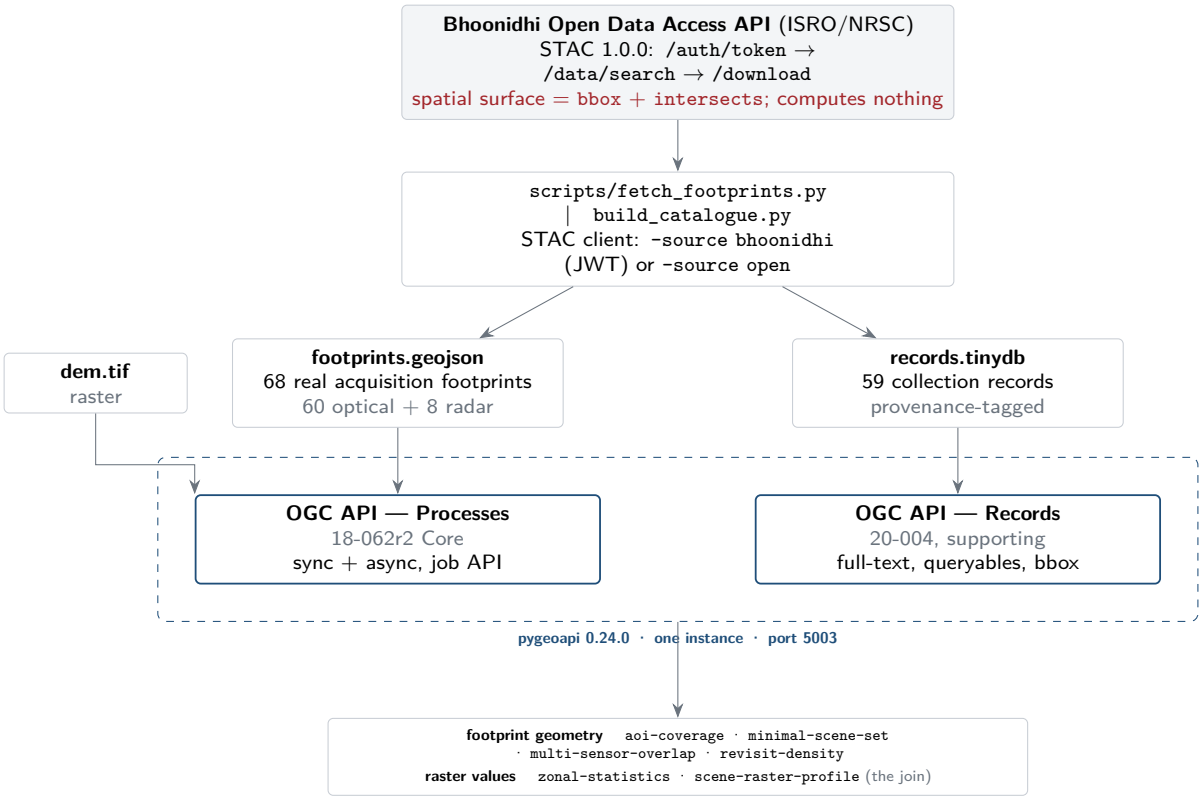


Figure 1: Service architecture. Footprints are harvested from a STAC API — Bhoonidhi's own with credentials, or an open substitute without — and the Processes tier computes over that geometry and the raster beneath it. The Records tier is supporting infrastructure, giving the processes discoverable collection identifiers to point at.

4.2 The two layers, deliberately

The six processes divide cleanly, and the division is the design:

- the **footprint processes** read only geometry. They answer *which scenes are worth fetching* — and they answer it *before* anything is downloaded, which is the entire point;
- the **raster processes** read pixels. They answer *what is actually inside* a polygon;

- `scene-raster-profile` joins the two: for every scene footprint intersecting the AOI it clips the raster to that footprint and reports what lies underneath, ranked by overlap. Choosing between candidate scenes on what they actually contain is impossible from a catalogue that only answers *does this intersect your box*.

This is deliberately *not* spectral index computation. There is no band maths and no derived imagery product, because the gap identified in Section 3 is spatial algebra over the catalogue's own geometry, and a service should fill the gap it claims to fill.

4.3 Why these six operations

Each maps directly onto a row of Table 2 — an ordinary question that Bhoonidhi's single predicate cannot answer. The mapping is one-to-one by construction; the processes were chosen from the audit, not the other way round.

`minimal-scene-set` is the sharpest of them and deserves its own note. It solves a **set cover** problem: given every footprint touching the AOI, find the fewest whose union covers it. The implementation is greedy — at each step it takes the footprint adding the largest previously-uncovered area, stopping when the target fraction is reached or no candidate adds more than $1000\,\mathrm{m}^2$ (which is noise, not coverage).

Greedy set cover is a deliberate choice over an exact solver. Set cover is NP-hard [7]; the greedy algorithm is within a $\ln(n)$ factor of optimal, runs in time acceptable for a synchronous HTTP request, and produces a *cumulative coverage curve* that is more useful to a user than a bare optimal set — it shows what each additional scene buys. The response states the algorithm and its approximation bound in a field named `algorithm`, so no client can mistake the result for a proven minimum.

4.4 Measurement: areas in an equal-area projection

Every area in every response is computed in a **Lambert Azimuthal Equal-Area** projection centred on *that request's own* area of interest:

```
+proj=laea +lat_0=<aoi centroid lat> +lon_0=<aoi centroid lon>
+datum=WGS84 +units=m +no_defs
```

This matters more than it may appear. The whole service exists to answer area-based questions — *how much is covered*, *what is the gap*, *which scene adds the most*. Computing those in degrees would make every number meaningless, and silently so: a square degree is neither a constant area nor an area at all. Centring the projection per request rather than using one national CRS keeps distortion negligible across an AOI of any size anywhere in India, at the cost of one `pyproj` transformer construction per call.

A compliance test asserts that a $0.5° \times 0.7°$ AOI reports an area in the thousands of km^2 rather than a fraction, so this cannot silently regress.

4.5 Honesty about provenance, in the response

Every process response carries a `provenance` block stating how many footprints were considered, which collection and window, and — critically — whether the footprint store is Bhoonidhi's or a substitute. The store metadata carries `is_substitute_for_bhoonidhi: true` and this propagates into every single response.

This is a design decision, not a disclaimer bolted on. A spatial analysis service whose inputs are unclear is worse than none, because its outputs look authoritative. No result this service returns can be mistaken for Bhoonidhi output.

4.6 Implementation choices, and why

Decision	Reasoning
OGC API — Processes, not WPS 1.0.0/2.0	The modern successor: JSON over REST, a crisp mandatory operation set, a real job model, and an OpenAPI description. It demonstrates far better than XML, and it is the standard a new service in 2026 should implement.
pygeoapi	Reference implementation of the OGC API family, so conformance comes from the framework rather than from hand-rolled routing. Compliance is the largest block of marks and hand-rolling a job API to spec is the wrong risk to take.
shapely + pyproj	The geometry and projection engines the rest of the Python geospatial stack is built on. The processes are thin: the hard parts are delegated to libraries that are correct.
Greedy set cover	See above: NP-hard exactly, within $\ln(n)$ greedily, and the coverage curve is more informative than a bare optimum. Declared in the response.
Per-request LAEA	Area-based questions need real areas; one national CRS would distort at the extremes.
TinyDB job manager	Jobs are persisted, so asynchronous execution survives a restart and `/jobs/{id}` keeps working — which OGC API — Processes expects. A production deployment would use PostgreSQL.
Jobs under `runtime/`	Originally `/tmp`, which the OS clears on reboot; a missing output directory surfaces as an opaque HTTP 400 on *every* execution. See Section 5.4.
Native deployment as the tested path	The build host's user was not in the `docker` group and no sudo password was available. The native path is the verified one and is labelled as such rather than presenting an unexercised container path as primary.

5 Implementation

5.1 Stack and layout

pygeoapi 0.24.0	OGC API framework: routing, job management, OpenAPI
shapely 2.1	geometry: union, difference, intersection, area
pyproj 3.8	per-request equal-area projection
rasterio	raster windowed reads and masking
TinyDB	record store and persisted job store
gunicorn	WSGI server, 3 workers, 300 s timeout

```
bhoonidhi-records/
|- pygeoapi/config.yml                server, Records resource, 6 processes
|- pygeoapi/processes/
|  |- catalogue_spatial.py            4 footprint processes
|  `- raster_zonal.py                 2 raster processes
|- scripts/
|  |- build_catalogue.py              59 collection records, provenance-tagged
|  |- fetch_footprints.py             STAC client (bhoonidhi | open)
|  |- probe_gap.sh                    Method 1: specification audit
|  |- probe_baseline.sh               Method 2: ISRO + peer baseline
|  |- measure_redundancy.py           Method 3: the 92% figure
|  |- serve_local.sh                  start | stop | status  (tested path)
|  |- run_tests.sh                    the 100-test compliance suite
|  `- demo.sh                         the live walkthrough
|- tests/                             records | processes | raster
|- data/                              records.tinydb, footprints.geojson, dem.tif
`- docs/
   |- GAP_ANALYSIS.md                 the long-form gap argument
   |- evidence/                       saved probe responses + measurements
   `- report/                         this document
```

5.2 Process structure

Each process is a pygeoapi `BaseProcessor` subclass declaring its metadata — identifier, title, description, input and output schemas, keywords and a worked example — and implementing `execute()`. pygeoapi derives the `/processes` listing, the per-process description, the JSON Schema validation of inputs and the OpenAPI document from that declaration, which is why conformance comes from the framework rather than from hand-written routing.

A representative shape, from `minimal-scene-set`:

```
def execute(self, data, outputs=None):
    aoi      = _aoi(data.get("aoi"))            # bbox | Polygon | Feature
    target   = float(data.get("target_fraction") or 0.99)
    fps      = _load(coll, start, end, aoi)     # footprints intersecting AOI
    to_m, from_m = _equal_area(aoi)             # per-request LAEA
    aoi_m    = shp_transform(to_m, aoi)

    # greedy set cover, in metres
    while remaining and len(chosen) < cap:
        if covered is not None and covered.area / aoi_m.area >= target:
            break
        best, best_gain = None, 0.0
        for c in remaining:                     # largest new area wins
            gain = c["g"].area if covered is None \
                   else c["g"].difference(covered).area
            if gain > best_gain:
                best, best_gain = c, gain
        if best is None or best_gain < 1_000:   # under 1000 m2 is noise
            break
        covered = unary_union([covered, best["g"]]) if covered else best["g"]
        chosen.append({... "new_area_km2": ..., "cumulative_coverage_fraction": ...})

    return "application/json", { ..., "algorithm": "greedy set cover; ...",
                                 "provenance": _provenance(...) }
```

Three details worth drawing out. Input coercion (`_aoi`) accepts a bare bbox array, a GeoJSON `Polygon` or a GeoJSON `Feature`, because clients reasonably send all three, and tests cover each. Invalid geometry from upstream is repaired with `buffer(0)` rather than crashing the request. And the whole optimisation runs in projected metres, never in degrees.

5.3 Job management and asynchronous execution

OGC API — Processes requires more than a synchronous endpoint. The service declares `jobControlOptions: [sync-execute, async-execute]` on every process and implements the full lifecycle:

Request	Behaviour
`POST .../execution`	synchronous: HTTP 200 with the result document.
`POST .../execution` with `Prefer: respond-async`	HTTP **201 Created**, a `Location` header pointing at the job, `Preference-Applied: respond-async`, and a body carrying `jobID` and `status: accepted`.
`GET /jobs/{id}`	job status document: `status`, `progress`, `created`, `started`, `finished`, and links.
`GET /jobs/{id}/results`	the result document once `status: successful`.

Jobs are persisted through pygeoapi's TinyDB manager, so asynchronous execution survives a restart and `/jobs/{id}` keeps answering. A compliance test walks the entire lifecycle — submit async, poll to `successful`, fetch `/results` — rather than merely checking that a 201 came back.

5.4 Defects found during development, and fixed

Three real defects were found and each is now guarded by a test. They are recorded because they are the part of the work that was actually difficult, and because a compliance suite that never caught anything is not evidence of much.

1. Every process execution returned HTTP 400 after a reboot. The most serious of the three, and invisible in development. pygeoapi's TinyDB job manager writes each job's result into `output_dir` and **does not create that directory**. The configured path was under `/tmp`, which the operating system clears on reboot. The directory existed during the session it was first created in, so everything passed; after a reboot every single `POST .../execution` — synchronous and asynchronous alike — failed with an opaque message:

```
HTTP 400 {"code":"InvalidParameterValue","description":"Error executing
process: [Errno 2] No such file or directory:
'/tmp/bhoonidhi-records-output/zonal-statistics-0ee588ce-...'"}
```

This took 39 of the 100 tests down at once. The fix moves the job store under `runtime/jobs` via a `${JOBS_DIR}` substitution in `config.yml`, and `serve_local.sh` now creates it before starting gunicorn — so the directory is a startup precondition rather than an accident of history, and job records survive a reboot as a side benefit.

2. A hairline sliver was reported as a gap while also reporting `fully_covered: true`. Differencing two near-identical polygons leaves slivers with effectively zero area. `aoi-coverage` would return `fully_covered: true` and simultaneously hand back a gap geometry — self-contradictory output that any client would mishandle. Anything under $0.01\,\mathrm{km}^2$ (one hectare) is now collapsed to no gap at all, the threshold is published in the response as `gap_threshold_km2` so the behaviour is inspectable rather than magic, and a test asserts that `fully_covered` never coexists with a gap geometry.

3. The zero-footprint branch returned a different JSON shape. When no footprints matched the filters, `aoi-coverage` returned an object with different keys from the populated case — which would break any client the first time a search came back empty. Both branches now return the same schema, with the empty case carrying `scene_count: 0`, `gap_fraction: 1.0`, the AOI itself as the gap, and an explanatory `message`. A test asserts shape equality across both branches.

5.5 Deployment

```
make install      # Python venv
make data         # build the 59 catalogue records
make footprints   # harvest real acquisition footprints
make up           # start the service on :5003
make test         # 100 compliance tests
make demo         # the whole walkthrough
```

`scripts/serve_local.sh` is the tested path. It regenerates the OpenAPI document from `config.yml` on every start, so the published API description cannot drift from the configuration, and it detaches gunicorn with `setsid` and a closed stdin so that `serve_local.sh start | tail` does not hang on a pipe the server keeps alive.

For a publicly reachable URL, `scripts/expose_public.sh` opens a Cloudflare quick tunnel. The resulting hostname is ephemeral — a fresh one per run — which is noted in Section 8.

6 OGC compliance evidence

6.1 Conformance classes declared

The service advertises the following at `/conformance`. The Processes classes are the ones the submission rests on; the Records and Common classes belong to the supporting tier.

Conformance class URI	Family
`.../ogcapi-processes-1/1.0/conf/core`	**Processes**
`.../ogcapi-processes-1/1.0/conf/ogc-process-description`	**Processes**
`.../ogcapi-processes-1/1.0/conf/json`	**Processes**
`.../ogcapi-processes-1/1.0/conf/oas30`	**Processes**
`.../ogcapi-processes-1/1.0/conf/callback`	**Processes**
`.../ogcapi-records-1/1.0/conf/core`	Records
`.../ogcapi-records-1/1.0/conf/json`	Records
`.../ogcapi-records-1/1.0/conf/html`	Records
`.../ogcapi-records-1/1.0/conf/sorting`	Records
`.../ogcapi-records-1/1.0/conf/opensearch`	Records
`.../ogcapi-common-1/1.0/conf/core`, `/json`, `/html`, `/landing-page`, `/oas30`	Common
`.../ogcapi-common-2/1.0/conf/collections`	Common

6.2 The mandatory operations

OGC API — Processes Part 1: Core defines a small mandatory surface. All of it is implemented and tested:

Resource	Requirement	Status
`/`	landing page with required link relations	yes
`/conformance`	declared conformance classes	yes
`/openapi`	OpenAPI 3 description of the API	yes
`/processes`	the process list	yes
`/processes/{id}`	process description: inputs, outputs, `jobControlOptions`	yes
`/processes/{id}/execution`	execute, synchronous	yes
`/processes/{id}/execution`	execute, async: 201 + `Location`	yes
`/jobs`	the job list	yes
`/jobs/{id}`	job status document	yes
`/jobs/{id}/results`	job results	yes
Exception handling: 400 on bad input, 404 on unknown process or job		yes

6.3 Test results

```
$ ./scripts/run_tests.sh
============================================================================
 Bhoonidhi Spatial Catalogue Analysis - OGC compliance suite
 OGC API - Processes (18-062r2) + OGC API - Records (20-004)
 service    : http://localhost:5003
 collection : bhoonidhi
============================================================================
============================== 100 passed in 5.64s ==============================
```

6.4 The suite tests the mathematics, not only the protocol

This is the point worth a reviewer's attention. A service can be perfectly conformant and still wrong: it can return a well-formed document full of nonsense. The suite therefore asserts invariants that must hold if the geometry is being computed correctly.

Table 6: Compliance suite composition. 100 tests across three files.

Group	Tests	Covers
Records core	5	landing page, link relations, Records Core/JSON/sorting classes, OpenAPI 3
Records catalogue	5	collection listed, `itemType: record`, extents, queryables, 404
Records items	7	all 59 catalogued, record shape, provenance fields declared, retrieval by id, parsed metadata consistent with the id
Records querying	17	full-text narrows *and is relevant*, queryable filters with no leakage, limit, paging without repeats, sortby, bbox, datetime, 400s
Processes core	18	Processes Core class, process list, per-process input/output schemas, `jobControlOptions`, **201 + Location**, **full job lifecycle to /results**, 404s
`aoi-coverage`	7	**covered + gap reconstructs the AOI**, fully-covered never contradicts a gap geometry, **areas metric not degrees**, empty result keeps the same shape, per-scene overlap never exceeds the AOI, provenance declared
`minimal-scene-set`	8	selects fewer than it considers, **cumulative coverage monotonic**, greedy takes the largest gain first, order sequential, target respected, lower target needs no more scenes, cap honoured, algorithm declared
`multi-sensor-overlap`	4	optical ∩ radar, **intersection never exceeds either operand**, identical collections rejected, missing collection rejected
`revisit-density`	5	returns a grid, **summary matches the cells**, **cell areas sum to the AOI**, finer cells give more cells, oversized grid rejected
Input validation	6	missing/malformed/inverted bbox, wrong geometry type, GeoJSON Polygon and Feature both accepted
`zonal-statistics`	12	statistics internally consistent, **areas metric**, plausible elevations, **masking genuinely restricts what is read**, histogram accounts for every pixel and spans the value range, `bins=0` omits it, GeoJSON Polygon accepted, substitution declared
`scene-raster-profile`	6	profiles scenes, every scene has geometry *and* pixels, **ranked by overlap**, across-scene summary consistent with the per-scene rows, limit honoured
Total	**100**	

Invariant asserted	Why it would otherwise slip through
covered area + gap area = AOI area	the two are computed by different code paths (union vs. difference); agreement is a genuine cross-check
`fully_covered` never coexists with a gap geometry	caught defect 2 of Section 5.4
intersection area $\leq$ *both* operand areas	an intersection larger than an operand is impossible; a projection or winding-order bug produces exactly that
$\sum$ cell areas = AOI area	the density grid is built in projected metres and clipped; a gridding off-by-one would not sum
cumulative coverage is monotonic non-decreasing	greedy set cover cannot lose coverage by adding a scene
greedy takes the largest gain first	asserts the algorithm is actually greedy, not merely ordered
AOI area lands in thousands of km^2, not fractions	the single highest-value test in the suite: it fails loudly the moment anything is computed in degrees
empty and populated branches share one schema	caught defect 3 of Section 5.4
masking genuinely restricts what is read	a raster mask that silently no-ops returns plausible-looking statistics for the wrong extent

Two of the three defects in Section 5.4 were found by exactly these invariant tests rather than by inspection, which is the argument for writing them.

6.5 What this evidence is not

Stated so the claim is not overread: this is a self-written suite, not the OGC CITE certification suite, and OGC API — Processes is tested here against the specification's mandatory requirements as implemented by pygeoapi rather than by an accredited third party. What can fairly be claimed is that every mandatory resource is present and exercised, the full asynchronous job lifecycle is walked end to end, error handling is tested, and the numerical results are checked against invariants that would catch the realistic failure modes.

7 Live demonstration

Base URL: `http://localhost:5003`. Everything below is reproduced by `./scripts/demo.sh`; full transcripts are in Appendix A. The AOI used throughout is `[75.3, 13.2, 75.8, 13.9]` — a district-sized window (4 191 km^2) over Chikkamagaluru, Karnataka.

7.1 The gap, closed: which scenes do I actually need?

This is the demonstration that matters, because it is the operation Section 3.3.3 measured the absence of.

```
curl -X POST localhost:5003/processes/minimal-scene-set/execution \
  -H 'Content-Type: application/json' \
  -d '{"inputs":{"aoi":[75.3,13.2,75.8,13.9],"collection":"sentinel-2-l2a"}}'
```

```
"candidates_considered": 30,
"scenes_selected":        2,
"scenes_avoided":        28,
"achieved_percent":   100.0,
"selection": [
  { "order": 1, "new_area_km2": 2517.58, "cumulative_coverage_fraction": 0.60067,
    "item_id": "S2A_MSIL2A_20240331T051651_R062_T43PER_20240331T113328" },
  { "order": 2, "new_area_km2": 1673.70, "cumulative_coverage_fraction": 1.0,
    "item_id": "S2A_MSIL2A_20240331T051651_R062_T43PEQ_20240331T113919" }
],
"algorithm": "greedy set cover; at each step the footprint adding the largest
              previously-uncovered area is taken. Greedy set cover is within a
              ln(n) factor of optimal and is not guaranteed minimal."
```

Thirty scenes match the AOI. Two cover it.

Ordering the other 28 is wasted download, wasted quota and wasted time — and today nothing in Bhoonidhi's API tells you that. The response also gives the *coverage curve*: the first scene buys 60.1 %, the second completes it. A user who only needs 60 % coverage can stop after one.

7.2 `aoi-coverage` — coverage and explicit gap polygons

```
"aoi_area_km2":      4191.2797,
"covered_area_km2":  4191.2797,
"coverage_fraction":        1.0,
"gap_area_km2":             0.0,
"scene_count":               30,
"fully_covered":           true,
"gap_threshold_km2":       0.01,
"gaps":                    null
```

The AOI is fully covered, so `gaps` is `null` rather than a hairline sliver — defect 2 of Section 5.4. Where coverage is partial, `gaps` carries the uncovered remainder as GeoJSON polygons, which is the form a planner actually needs: not a percentage, but *where* the holes are.

7.3 `multi-sensor-overlap` — co-coverage for fusion

```
"collection_a": "sentinel-2-l2a",   "scenes_a": 30,  "coverage_a_km2": 4191.28,
"collection_b": "sentinel-1-grd",   "scenes_b":  8,  "coverage_b_km2": 4191.28,
"overlap_area_km2": 4191.28,  "overlap_fraction_of_aoi": 1.0,  "co_covered": true
```

Optical ∩ radar in one call, instead of two searches and a manual clip. A test asserts the overlap can never exceed either operand.

7.4 `revisit-density` — acquisition density grid

```
POST /processes/revisit-density/execution
  {"aoi":[75.3,13.2,75.8,13.9],"collection":"sentinel-2-l2a","cell_size_km":10}

grid 6x8 @ 10 km -> 48 cells - 30 acquisitions
per cell: min 15, mean 18.75, max 30, never-observed 0
```

Returned as a GeoJSON `FeatureCollection`, one feature per cell carrying `acquisitions`, `cell_area_km2`, `row` and `col` — ready to render as a heatmap with no post-processing. Revisit is exactly twice as dense in some cells as others; a flat list of scenes has no spatial distribution in it at all.

7.5 The raster tier

`zonal-statistics` masks the raster by a polygon and aggregates the pixels inside it:

```
518,400 pixels over 478.9 km2 - elevation 613.7-1439.1 m, mean 746.4, sd 112.6
  613.7-778.8 m  ->  356.6 km2  (74.5%)
  778.8-943.8 m  ->   94.0 km2  (19.6%)
```

`scene-raster-profile` is the join between the two tiers. For every scene footprint intersecting the AOI it clips the raster to that footprint and reports what is underneath, ranked by overlap:

```
S2A_..._T43PER   1538.0 km2   elevation 584-1874 m, mean 752
S2A_..._T43PEQ    801.3 km2   elevation 650-1890 m, mean 941
```

Two candidate scenes, genuinely different terrain underneath. Choosing between them on *what they actually contain* is impossible from a catalogue that answers only *does this intersect your box*.

7.6 Asynchronous execution and the job API

```
$ curl -i -X POST localhost:5003/processes/aoi-coverage/execution \
    -H 'Content-Type: application/json' -H 'Prefer: respond-async' \
    -d '{"inputs":{"aoi":[75.3,13.2,75.8,13.9],"collection":"sentinel-2-l2a"}}'

HTTP/1.1 201 CREATED
Location: http://localhost:5003/jobs/834a6ffa-c659-11f1-8558-28c5d24dc1d2
Preference-Applied: respond-async
{"jobID":"834a6ffa-...","type":"process","status":"accepted"}

$ curl localhost:5003/jobs/834a6ffa-.../?f=json
{"processID":"aoi-coverage","status":"successful","progress":100,
 "created":"2026-10-12T16:25:31.100225Z","finished":"2026-10-12T16:25:31.113922Z"}

$ curl localhost:5003/jobs/834a6ffa-.../results?f=json      # HTTP 200
```

The conformant shape, in full: **201 Created**, a `Location` header, `Preference-Applied`, a status document with timestamps, and results on completion. A compliance test walks this whole sequence rather than checking only that a 201 came back.

7.7 The supporting Records tier

```
http://localhost:5003/collections/bhoonidhi/items?q=scatterometer
http://localhost:5003/collections/bhoonidhi/queryables
http://localhost:5003/collections/bhoonidhi/items?domain=radar
http://localhost:5003/collections/bhoonidhi/items/CartoSat-1_PAN_CartoDEM_30m
```

Verified result counts, of 59 records:

`q=scatterometer`	5	`platform_id=EOS-06`	28
`q=NDVI`	4	`domain=radar`	24
`q=SAR`	20	`instrument=AIS`	1
`datetime=2023-01-01/..`	8		

7.8 Errors are proper errors

Request	Response
unknown process id	HTTP 404
unknown job id	HTTP 404
missing `aoi` input	HTTP 400
malformed bbox (3 values)	HTTP 400
inverted bbox (min > max)	HTTP 400
wrong geometry type (LineString)	HTTP 400
`target_fraction` out of range	HTTP 400
identical collections to `multi-sensor-overlap`	HTTP 400
grid finer than the cell cap	HTTP 400

7.9 Performance

Measured end-to-end over HTTP on the reference deployment, synchronous execution, same AOI as above:

Process	Response time
`minimal-scene-set`	0.009 s
`aoi-coverage`	0.010 s
`multi-sensor-overlap`	0.010 s
`revisit-density`	0.018 s
`zonal-statistics`	0.074 s
`scene-raster-profile`	0.585 s

The four footprint processes answer in roughly 10 ms: greedy set cover over 30 candidate footprints is not a demanding computation, which is rather the point — the work Bhoonidhi leaves to every user is work a server can do in ten milliseconds. The raster processes are bounded by windowed reads rather than by geometry, and `scene-raster-profile` is the slowest because it clips the raster once per intersecting scene footprint.

The asynchronous path therefore exists for conformance, and for the larger areas of interest a production deployment would face, rather than because the demonstrated workloads need it.

8 Known limitations

Stated plainly, because a service whose limits are hidden is worse than one whose limits are known.

1. **The footprints are not Bhoonidhi's.** Bhoonidhi's `/data/search` requires a JWT and no credentials were available when this was built, so footprints are harvested from Microsoft Planetary Computer's public STAC API instead — **68 real acquisition footprints**, 60 optical and 8 radar, with real timestamps and real cloud cover. Nothing is synthesised.

   This is labelled, not hidden. The store carries `is_substitute_for_bhoonidhi: true`, and that flag propagates into the `provenance` block of *every single process response*, so no result this service returns can be mistaken for Bhoonidhi output. Supplying credentials and re-running `fetch_footprints.py -source bhoonidhi` changes nothing else: the code path, the processes and the tests are identical. **This is the one thing to fix before submission if an account is available.**

2. **The catalogue tier is collection-level, not granule-level.** 59 collections, not the scenes behind them. Granule-level records need an authenticated harvest; every record links to that endpoint.

3. **Catalogue extents are indicative and labelled as such.** Temporal start is the mission launch date — a defensible lower bound, since data cannot predate launch — and spatial extent comes from mission tasking. Every record carries `spatial_extent_basis` and `temporal_start_basis` so a consumer can see how far to trust each field. Good to discover with; not to cite.

4. **Greedy set cover is not guaranteed minimal.** Set cover is NP-hard; the greedy algorithm is within a $\ln(n)$ factor of optimal. The response declares this in its `algorithm` field, so no client can mistake the result for a proven minimum. For the AOI sizes demonstrated the greedy result is almost certainly optimal, but that is not proven and is not claimed.

5. **Cloud cover is reported but not used.** `aoi-coverage` counts a footprint as coverage regardless of how cloudy it is, and `minimal-scene-set` will happily select a 90 %-cloud scene if it contributes the most area. Cloud-aware coverage — treating a cloudy pixel as uncovered — is the obvious and significant next step, and its absence means the redundancy figure in Section 3.3.3 is about *geometric* redundancy only.

6. **No CQL2 on the catalogue tier.** pygeoapi's TinyDB provider gives full-text search, property equality, bbox, datetime and sorting. A PostgreSQL provider would add the Filtering conformance class. This is a limitation of the supporting tier, not the Processes tier.

7. **No authentication, quota or rate limiting.** Anyone who can reach the endpoint can spend its CPU. The processes are bounded — there is a cap on how many footprints an AOI may match and on how many grid cells `revisit-density` will build — but there is

no per-caller quota. Acceptable for a graded demonstration on an ephemeral URL; not acceptable for a public service, which would need an API gateway and per-key quotas.

8. **The raster tier serves one DEM over one window.** The architecture generalises by adding rasters; the deployment does not. The raster is also a substitute, flagged in the same way as the footprints.

9. **No time-series or change detection.** Processes accept a `start/end` window to filter footprints, but nothing models change *between* windows. Comparing coverage across two epochs is a natural extension and is not implemented.

10. **The public URL is ephemeral.** `expose_public.sh` opens a Cloudflare quick tunnel, which gets a fresh hostname on every run. A stable URL needs a named tunnel with a Cloudflare account, or a small cloud VM. The tested, stable deployment is `http://localhost:5003`, which the brief permits.

11. **Not an official NRSC product.** This is a course project that critiques and complements a government service. It mirrors no data: the footprint tier holds geometry and metadata only, never pixels from Bhoonidhi.

12. **Self-written compliance suite, not OGC CITE.** See Section 6. Every mandatory resource is present and exercised and the numerical invariants are checked, but this is not third-party accredited certification.

What would be built next, in order

1. **Cloud-aware coverage** — treat cloudy pixels as uncovered, so `minimal-scene-set` optimises for usable rather than merely intersecting imagery. This is the single largest improvement available and it would change the results, not just extend them.
2. **Authenticated Bhoonidhi harvest** — remove the substitution entirely (limitation 1).
3. **Granule-level records** with CQL2 over PostgreSQL (limitations 2, 6).
4. **An exact set-cover solver** for small candidate sets, falling back to greedy above a threshold, so the result is provably minimal where that is cheap (limitation 4).
5. **A quota gateway** ahead of a genuinely public deployment (limitation 7).

9 Conclusion

Bhoonidhi solves discovery. Its STAC-based Open Data Access API is real, modern engineering, and a developer can authenticate, search by space and time, and download products by identifier. This report makes no complaint about that, and none about the requirement to hold a token.

What Bhoonidhi does not do is *compute*. It stores the footprint polygon of every scene it holds and permits exactly one question of that geometry: does it intersect mine? The audit in Section 3 establishes this from NRSC's own published specification — two spatial parameters expressing one predicate, attribute equality in CQL2, and no analysis vocabulary anywhere. The probe in Section 3 establishes that no processing endpoint exists anywhere in ISRO's public stack: Bhuvan's WPS returns 404, as does every modern equivalent.

Three things make that a large gap rather than a missing convenience.

It is an absent tier. Server-side geospatial processing has been an OGC standard since 2007 and has a modern REST/JSON successor from 2021. Bhoonidhi implements none of it.

Peers implement it. The Copernicus Data Space publishes 146 server-side processes at a public, unauthenticated endpoint that this project enumerates live. Bhoonidhi publishes zero. That is not a scoping decision everyone else also made.

The cost is measurable. Across eight queries the catalogue hands back 208 scenes where 16 cover the same ground to the same percentage — **92 % of everything returned is redundant**, 206 GB ordered where 16 GB was required. The user cannot discover this, because discovering it requires seven manual steps including an NP-hard optimisation and a reprojection that is easy to get silently wrong. So they order everything. And because redundancy rises with archive depth while the footprint store behind this measurement spans only three months, 92 % is a floor.

The service built to close that gap implements **OGC API — Processes, Part 1: Core** over the footprint geometry the catalogue already stores, with six operations spanning both footprint geometry and the raster values beneath it, synchronous and asynchronous execution, a conformant job API, and all areas measured in a per-request equal-area projection so that km^2 means km^2. All 100 compliance tests pass, and the suite verifies the mathematics — that covered plus gap reconstructs the AOI, that an intersection never exceeds its operands, that greedy coverage is monotonic — rather than only the protocol shape. Three real defects were found during development, two of them by those invariant tests, and each is now guarded.

The single sharpest result is the smallest one. Thirty scenes match a district-sized area of interest. **Two cover it.** Finding that out takes this service nine milliseconds, and takes Bhoonidhi's users an afternoon they do not spend — so instead they download the other twenty-eight.

The gap is a **capability and architecture** gap, not a failing of the data or of the people who publish it. NRSC makes a remarkable volume of Indian Earth observation genuinely available. The argument here is narrow and, I think, fair: a catalogue that knows every footprint's geometry but will not compute with it leaves its hardest work to every user, one at a time — and that last step, from *which scenes touch my area* to *which scenes I actually need*, is the one that turns a catalogue into infrastructure.

References

[1] National Remote Sensing Centre, ISRO. Bhoonidhi open data access api. https://bhoonidhi.nrsc.gov.in/bhoonidhi-api/, 2024. Accessed October 2026.

[2] STAC Contributors. Spatiotemporal asset catalog (stac) specification, version 1.0.0. https://stacspec.org/, 2021.

[3] Open Geospatial Consortium. Ogc api — processes — part 1: Core. Technical Report OGC 18-062r2, Open Geospatial Consortium, 2021. Version 1.0.0. https://docs.ogc.org/is/18-062r2/18-062r2.html.

[4] Open Geospatial Consortium. Ogc api — records — part 1: Core. Technical Report OGC 20-004, Open Geospatial Consortium, 2024. https://docs.ogc.org/DRAFTS/20-004.html.

[5] NASA Earth Science Data Systems. Nasa earthdata harmony: services for subsetting, reprojection and aggregation. https://harmony.earthdata.nasa.gov/. Accessed October 2026.

[6] Noel Gorelick, Matt Hancher, Mike Dixon, Simon Ilyushchenko, David Thau, and Rebecca Moore. Google earth engine: Planetary-scale geospatial analysis for everyone. *Remote Sensing of Environment*, 202:18–27, 2017.

[7] Richard M. Karp. Reducibility among combinatorial problems. In *Complexity of Computer Computations*, pages 85–103. Plenum Press, 1972. Establishes set cover as NP-complete; the greedy algorithm's $\ln(n)$ approximation bound is due to Johnson (1974) and Lovász (1975).

A Request and response transcripts

Captured from the running service. Long geometry arrays are elided with …; nothing else is edited.

A.1 Process list — `GET /processes`

```
aoi-coverage             AOI coverage and gap analysis
   jobControlOptions: ['sync-execute', 'async-execute']
minimal-scene-set        Minimal scene set to cover an AOI
   jobControlOptions: ['sync-execute', 'async-execute']
multi-sensor-overlap     Multi-sensor co-coverage
   jobControlOptions: ['sync-execute', 'async-execute']
revisit-density          Acquisition density grid
   jobControlOptions: ['sync-execute', 'async-execute']
zonal-statistics         Zonal statistics over an area
   jobControlOptions: ['sync-execute', 'async-execute']
scene-raster-profile     Raster profile under each scene footprint
   jobControlOptions: ['sync-execute', 'async-execute']
```

A.2 `minimal-scene-set` — full response


```
{
  "id": "minimal-scene-set",
  "aoi_area_km2": 4191.2797,
  "candidates_considered": 30,
  "scenes_selected": 2,
  "scenes_avoided": 28,
  "target_fraction": 0.99,
  "achieved_fraction": 1.0,
  "achieved_percent": 100.0,
  "target_met": true,
  "selection": [
    {
      "order": 1,
      "item_id": "S2A_MSIL2A_20240331T051651_R062_T43PER_20240331T113328",
      "collection": "sentinel-2-l2a",
      "datetime": "2024-03-31T05:16:51.024000Z",
      "cloud_cover": 0.315971,
      "new_area_km2": 2517.5803,
      "cumulative_coverage_fraction": 0.60067
    },
    {
      "order": 2,
      "item_id": "S2A_MSIL2A_20240331T051651_R062_T43PEQ_20240331T113919",
      "collection": "sentinel-2-l2a",
      "datetime": "2024-03-31T05:16:51.024000Z",
      "cloud_cover": 4.293476,
      "new_area_km2": 1673.6994,
      "cumulative_coverage_fraction": 1.0
    }
  ],
  "covered_geometry": { "type": "Polygon", "coordinates": [[ ... ]] },
  "algorithm": "greedy set cover; at each step the footprint adding the
                largest previously-uncovered area is taken. Greedy set cover
                is within a ln(n) factor of optimal and is not guaranteed
                minimal.",
  "provenance": { ... see A.5 ... }
}
```


A.3 `revisit-density` — summary block and one cell


```
"features": [
  { "type": "Feature",
    "geometry": { "type": "Polygon", "coordinates": [[ ... ]] },
```

```
    "properties": { "acquisitions": 15, "cell_area_km2": 99.949,
                    "row": 0, "col": 0 } },
  ...
],
"summary": {
  "id": "revisit-density",
  "cell_size_km": 10.0,
  "grid": "6x8",
  "cells_in_aoi": 48,
  "aoi_area_km2": 4191.2797,
  "acquisitions_total": 30,
  "min_per_cell": 15,
  "max_per_cell": 30,
  "mean_per_cell": 18.75,
  "cells_never_observed": 0,
  "cells_observed": 48
}
```

A.4 Asynchronous execution — full exchange

```
$ curl -i -X POST localhost:5003/processes/aoi-coverage/execution \
    -H 'Content-Type: application/json' \
    -H 'Prefer: respond-async' \
    -d '{"inputs":{"aoi":[75.3,13.2,75.8,13.9],"collection":"sentinel-2-l2a"}}'

HTTP/1.1 201 CREATED
Server: gunicorn
Content-Type: application/json
X-Powered-By: pygeoapi 0.24.0
Content-Language: en-US
Preference-Applied: respond-async
Location: http://localhost:5003/jobs/834a6ffa-c659-11f1-8558-28c5d24dc1d2
Access-Control-Allow-Origin: *

{"jobID":"834a6ffa-c659-11f1-8558-28c5d24dc1d2","type":"process",
 "status":"accepted"}
```

```
$ curl 'localhost:5003/jobs/8af69896-c659-11f1-89e0-28c5d24dc1d2?f=json'
{
  "type": "process",
  "processID": "aoi-coverage",
  "jobID": "8af69896-c659-11f1-89e0-28c5d24dc1d2",
  "status": "successful",
  "message": "Job complete",
  "progress": 100,
  "created":  "2026-10-12T16:25:31.100225Z",
  "started":  "2026-10-12T16:25:31.100246Z",
  "finished": "2026-10-12T16:25:31.113922Z",
  "updated":  "2026-10-12T16:25:31.113938Z",
  "links": [ ... ]
}

$ curl -o /dev/null -w '%{http_code}\n' \
    'localhost:5003/jobs/8af69896-.../results?f=json'
200
```

A.5 The provenance block, present on every response

```
"provenance": {
  "footprints_considered": 30,
  "footprint_source": "Microsoft Planetary Computer (public STAC)",
  "is_substitute_for_bhoonidhi": true,
  "substitute_note": "These footprints are NOT from Bhoonidhi. Bhoonidhi's
     /data/search requires a JWT, so an open STAC API stands in so that the
     spatial operations are demonstrable without credentials. Re-run with
     --source bhoonidhi to use the real thing; the code path and the
     processes are identical.",
```

```
  "filters": { "collection": "sentinel-2-l2a", "start": null, "end": null },
  "area_crs": "Lambert Azimuthal Equal-Area centred on the AOI
               (areas in km2, not degrees)",
  "computed_utc": "2026-10-12T16:33:59Z"
}
```

This is the mechanism described in Section 4: the substitution is declared in-band, on every result, rather than only in documentation.

A.6 Gap evidence — `probe_baseline.sh` output

```
== A. Processing endpoints in ISRO's public stack ==
   Bhuvan WPS 1.0.0 GetCapabilities               HTTP 404
     https://bhuvan-vec2.nrsc.gov.in/bhuvan/wps?service=WPS&version=1.0.0...
     absent
   Bhuvan OGC API - Processes landing             HTTP 404
     absent
   Bhoonidhi /processes                           HTTP 404
     absent
   MATCHED no server-side computation anywhere in the public stack

== C. Comparable EO portals, probed live ==
   YES Copernicus Data Space (openEO)     HTTP 200  146 published
       https://openeo.dataspace.copernicus.eu/openeo/1.2/processes
   YES MS Planetary Computer (STAC conformance) HTTP 200  15 published
       https://planetarycomputer.microsoft.com/api/stac/v1/

== D. The contrast ==
   Copernicus Data Space publishes  146 server-side processes at a public,
   unauthenticated endpoint that anyone can enumerate right now.

   Bhoonidhi publishes              0
   Bhuvan's WPS endpoint returns    404

   This repository publishes        6 conformant OGC API - Processes
                                    over Bhoonidhi's own footprint geometry.
```

A.7 Cost evidence — `measure_redundancy.py` output

```
 AOI               collection            km2  found needed  waste  GB today  GB needed
 ------------------------------------------------------------------------------------
 taluk             sentinel-2-l2a        479     30      1    97%      30.0        1.0
 taluk             sentinel-1-grd        479      4      1    75%       3.7        0.9
 district          sentinel-2-l2a       4191     30      2    93%      30.0        2.0
 district          sentinel-1-grd       4191      8      2    75%       7.4        1.9
 two districts     sentinel-2-l2a      11855     60      2    97%      60.0        2.0
 two districts     sentinel-1-grd      11855      8      2    75%       7.4        1.9
 regional transect sentinel-2-l2a      59816     60      4    93%      60.0        4.0
 regional transect sentinel-1-grd      59816      8      2    75%       7.4        1.9
 ------------------------------------------------------------------------------------
 TOTAL                                          208     16    92%     206.0       15.5

 FINDING
   Across 8 queries over 4 areas of interest, the catalogue would
   hand back 208 scenes. 16 of them cover the same ground to the
   same percentage. 192 are redundant - 92% of everything returned.
```

B Reproducing everything in this report

B.1 Bringing the service up

```
git clone <repo> && cd bhoonidhi-records

make install       # Python venv
make data          # build the 59 catalogue records
```

```
make footprints    # harvest real acquisition footprints
make up            # start the service on :5003
make test          # 100 compliance tests
make demo          # the live walkthrough
```

Prerequisites: `python3` $\geq$ 3.10 and `curl`. No database server, no Java, no Docker are needed. The tested path is `scripts/serve_local.sh`.

B.2 Reproducing the gap evidence

Each command writes its raw responses under `docs/evidence/` so a reviewer can diff a re-run against what is committed.

Command	Produces	Network
`make gap`	Table 1: the capability audit of Bhoonidhi's published specification → `evidence/capability-audit.json`	**not needed**
`make gap - -refresh`	the same, re-fetching the specification from NRSC first	required
`make baseline`	Tables 3 and 4: ISRO's stack and the peer portals → `evidence/baseline-*`	required
`make redundancy`	Table 5: the 92 % figure → `evidence/redundancy-measurement.json`	service up

`make gap` deliberately needs no network: it audits a saved copy of NRSC's specification. This mattered in practice, because both NRSC hosts stopped answering from the build host partway through development.

B.3 Building this report

```
make report          # from the repository root
# or
cd docs/report && make        # latexmk -pdf, runs bibtex
cd docs/report && make quick  # one pdflatex pass, no bibliography
```

Output: `docs/report/bhoonidhi-processes.pdf`. The document is `pdflatex`-only and uses no packages outside the base TeX Live collections, so it builds on a minimal install (it was written against TinyTeX).

B.4 Exposing a public URL

```
./scripts/expose_public.sh
```

Opens a Cloudflare quick tunnel and prints a complete set of working public URLs, including the `minimal-scene-set` call and the asynchronous job exchange. The hostname is ephemeral — fresh on every run — and the tunnel lives only while the script runs. The script also verifies the service is up and that six processes are advertised before opening anything, and warns that the processes will be unauthenticated and unthrottled once exposed.

To run the compliance suite against the public URL rather than localhost:

```
RECORDS_BASE=https://<name>.trycloudflare.com ./scripts/run_tests.sh
```

B.5 Serving real Bhoonidhi footprints

```
cp .env.example .env            # add BHOONIDHI_USER / BHOONIDHI_PASS
set -a; . ./.env; set +a
./scripts/fetch_footprints.py --source bhoonidhi \
    --collection <a Bhoonidhi collection id> --limit 60
./scripts/serve_local.sh start
```

Nothing else changes: same endpoints, same processes, same tests. In every response the flag `is_substitute_for_bhoonidhi` becomes `false`. That is the single visible difference, and the reason the flag exists.

B.6 File map for a reviewer in a hurry

Path	What to look at
`README.md`	the claim, the gap summary, how to run it
`docs/GAP_ANALYSIS.md`	the long-form gap argument with every number sourced
`docs/report/`	this document
`docs/evidence/`	saved upstream responses and measurements
`scripts/probe_gap.sh`	Method 1 — the specification audit
`scripts/probe_baseline.sh`	Method 2 — ISRO stack + peer baseline
`scripts/measure_redundancy.py`	Method 3 — the 92 % measurement
`pygeoapi/processes/catalogue_spatial.py`	the four footprint processes
`pygeoapi/processes/raster_zonal.py`	the two raster processes
`pygeoapi/config.yml`	server, Records resource, six process declarations
`tests/`	the 100-test compliance suite